

AI OPERATIONS PORTFOLIO · USER MANUAL

Agent Operations Framework

A multi-agent system organized like a company — that extends and improves itself.

John White

AI Operations & Optimization

[linkedin.com/in/johnwhiteimse](https://www.linkedin.com/in/johnwhiteimse) · github.com/johnwhite-ai/ai-operations-portfolio

Contents

- 01** Executive Summary
- 02** The Problem
- 03** Architecture: A Company, Not a Toolbox
- 04** The Differentiator: It Grows Its Own Capabilities
- 05** How a Request Flows Through the System
- 06** Design Rationale & Tradeoffs
- 07** What This Project Demonstrates

01 Executive Summary

The Agent Operations Framework is a multi-agent system modeled on how a high-performing company is organized: five specialist departments and twelve agents, coordinated by an orchestrator acting as the executive function. Every output is sourced. And when the system hits a capability it lacks, it does not fail — it finds or builds that capability and keeps it.

It was built in production for a Solutions Engineering team, where it standardized how customer engagements ran regardless of who owned the account. This is a clean-room, domain-neutral rebuild of that pattern.

THE ONE-LINE VERSION

Most agent systems are a flat pile of tools that break the moment the work changes. This one is structured like a company and grows its own capabilities — the difference between automating a task and operating a function.

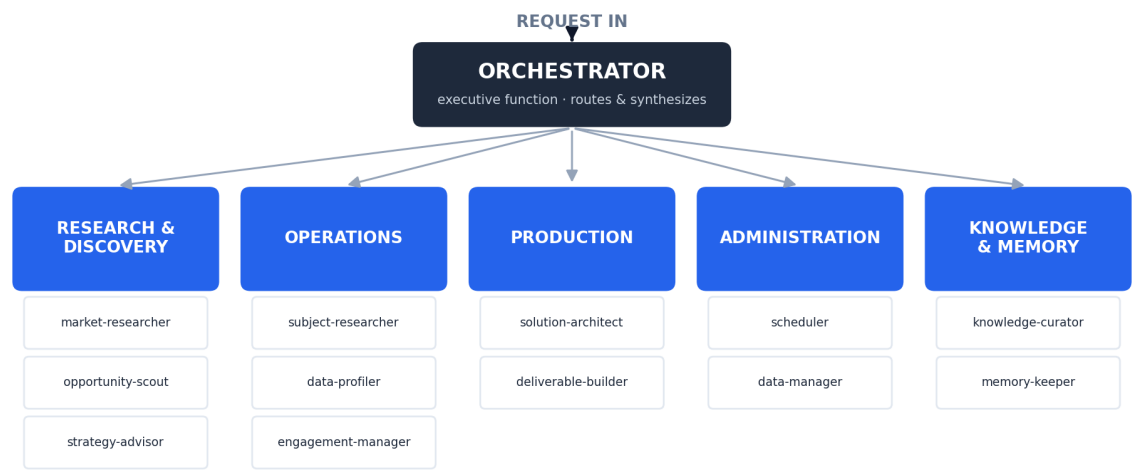
02 The Problem

In most operations teams, complex recurring work — researching an account, profiling a dataset, compiling a brief — is done differently by every person, every time. Quality depends on who did it and how much time they had. The output is rarely sourced, so no one downstream knows which numbers to trust. And the knowledge lives in people's heads, so when someone leaves, the capability leaves with them.

The result is inconsistent output, untrustworthy data, and an operation that cannot scale without scaling headcount. Most attempts to fix this with AI produce a brittle single agent that works in the demo and breaks the first time the real work shifts.

03 Architecture: A Company, Not a Toolbox

The framework treats a complex workflow the way a well-run organization does: an executive routes work to specialist departments, each owning one function and doing it the same way every time. The structure was researched against how high-performing companies actually divide labor, then encoded into the agent hierarchy.



12 specialist agents · each excellent at one job · coordinated like a company

The orchestrator routes a request across five specialist departments; twelve agents, each excellent at one job.

The orchestrator (executive function)

Interprets the request, decomposes it into tasks, routes each to the right department, manages hand-offs as structured context (never raw text), and synthesizes the result. It enforces the standards on every output: sourced, first-principles, decision-useful, honest about gaps.

The five departments

Department	Owns	Agents
Research & Discovery	Research, opportunity sizing, capability scouting	market-researcher, opportunity-scout, strategy-advisor
Operations	Subject research, data profiling, driving the work	subject-researcher, data-profiler, engagement-manager
Production	Solution design and deliverable building	solution-architect, deliverable-builder
Administration	Scheduling, data hygiene, catalog ownership	scheduler, data-manager
Knowledge & Memory	Capturing lessons, recalling history	knowledge-curator, memory-keeper

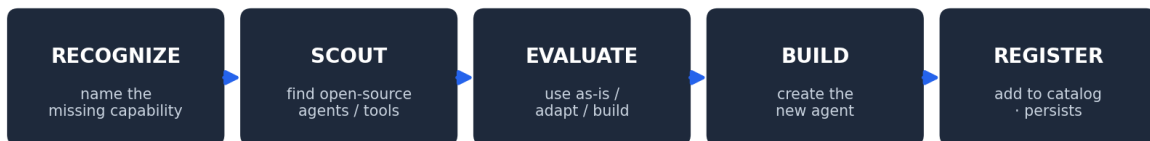
04 The Differentiator: It Grows Its Own Capabilities

This is what sets the framework apart from a typical agent build, and it came directly from a real roadblock: in production, the agents worked at launch, then degraded as new products and customers arrived with needs no existing agent covered. The usual fix is a human noticing and rewriting prompts — the cycle that gets internal AI tools abandoned.

Instead, the framework runs the Capability Gap Protocol: when no agent can handle a task, the system acquires the capability rather than failing.

THE CAPABILITY GAP PROTOCOL

when no agent can do the job, the system grows one instead of failing



the capability is now permanent — the system is better than it was an hour ago

The Capability Gap Protocol: recognize, scout, evaluate, build, register — the new capability persists.

HOW THE PROTOCOL WORKS

1) Recognize the gap and state it plainly. 2) Dispatch the opportunity-scout to find existing open-source agents or tools that fill it. 3) Evaluate: use as-is, adapt, or build from scratch. 4) Build or adapt the new agent in the standard file structure. 5) Register it in the catalog so the capability persists for every future request.

HOW MOST PEOPLE BUILD

ONE AGENT

or a fixed prompt chain

*static · breaks when scope changes**needs a human to extend it*

THIS SYSTEM

COMPANY OF AGENTS

that extends itself

self-extending · self-improving**grows its own capabilities***The contrast: a static single agent vs. a self-extending company of agents.*

WHY IT MATTERS

A normal automation breaks when scope changes and waits for a human to extend it. This system treats a capability gap as a routine event with a defined response. It is the difference between automation that survives contact with a changing business and automation that does not.

05 How a Request Flows Through the System

A worked example, illustrative: "Profile this new account's raw operations data and tell me the three biggest risks. They sent a PDF in a format I've never seen."

- **Orchestrator** interprets intent — profile data, assess risk, deliver a sourced summary — and routes to Operations.
- **data-profiler** begins, but the PDF table format is one no current agent can parse. It reports a capability gap.
- **Capability Gap Protocol fires:** opportunity-scout finds an open-source PDF-table extractor; it is usable with a thin wrapper; a new agent is created and registered in the catalog.
- **Work resumes:** data-profiler returns a clean, sourced input set, every field tagged Confirmed / Estimated / Missing.
- **deliverable-builder** assembles the top-three-risks summary, each risk sourced and confidence-tagged.
- **Knowledge & Memory** captures the lesson: this format is now known, and the new extractor is available next time.

THE POINT

The system did not fail when it hit an unknown format. It recognized the gap, acquired the capability, finished the job, and is now permanently better at it.

06 Design Rationale & Tradeoffs

Why specialist agents instead of one capable generalist?

One agent asked to research, analyze, and write does all three adequately and none excellently. Splitting the work into specialists, each with one job and one standard, makes each step consistent and the whole chain auditable. The cost is more coordination logic; the payoff is reliability and the ability to improve one agent without destabilizing the rest.

Why model it on a company structure?

Companies converged on the department-and-executive pattern because it scales division of labor under changing demands. Borrowing that structure gave the system a proven template for routing, escalation, and ownership — rather than inventing an agent topology from scratch.

Why keep a human in the loop on self-modification?

The system proposes new agents and flags low-confidence runs, but does not silently rewrite itself. Trust requires that the system never changes its own behavior or overstates its certainty behind the user's back. Self-extension with human approval is powerful; silent self-modification is a liability.

07 What This Project Demonstrates

Capability	How this project proves it
Multi-agent system design	An orchestrator coordinating five departments and twelve specialist agents with clean hand-offs
AI workflow architecture	A repeatable process that runs consistently regardless of operator
Trustworthy AI output	The Confirmed / Estimated / Missing sourcing discipline on every result
Creative problem-solving under AI roadblocks	Self-extending agents via the Capability Gap Protocol
Operational judgment	Built around how work actually breaks, not around the tooling
Knowledge capture	Lessons encoded into the system, not lost when a person leaves

THE TAKEAWAY

When AI hits a wall, the instinct on display here is to redesign the system around the wall instead of abandoning it. That is the difference between someone who uses AI tools and someone who builds AI operations.